sinSPICE

Synthetic Ultraviolet Spectral-Line Emission from 3D MURaM Simulations

Code Review & Functional Description

July 22, 2026

1 Purpose

sinSPICE is a research code that turns three-dimensional radiative-MHD simulations of the solar atmosphere into *synthetic ultraviolet emission-line spectra*, of the kind observed by instruments such as Solar Orbiter/SPICE (from which the name derives). Given a MURaM simulation snapshot, it produces wavelength-resolved intensity maps of a chosen transition-region or coronal emission line, for a time series of snapshots.

Concretely, the pipeline performs four steps:

1. **Read** temperature, electron density and velocity cubes from MURaM binary output.
2. **Build** a CHIANTI contribution function $G(T, n_e)$ for a selected ion and spectral line.
3. **Integrate** a Doppler-shifted, thermally broadened Gaussian line profile through every vertical column of the model atmosphere.
4. **Repeat** across many timesteps (in parallel) and save the resulting intensity cube to disk.

The emergent product is an array of shape $(\, n_{\text{time}},\, n_x,\, n_y,\, n_\lambda \,)$: for each snapshot, an image of the Sun in the chosen line, resolved in wavelength at every pixel.

2 Repository Layout

The tracked source is deliberately small; the heavy lifting is delegated to NumPy, SciPy, ChiantiPy and joblib.

Path	Role
`modules/muram.py`	Low-level readers for MURaM binary formats (full cubes, sub-cubes, slices, columns, intensities). Memory-maps 3D data.
`modules/dataLoader.py`	`MURaMData`: application-level wrapper exposing `Temp`, `ne`, `vz` for one snapshot.
`modules/synthesizeLines.py`	The physics core: `spectralLine` ($G(T, n_e)$ tables) and `spectrum` (1D/2D line synthesis).
`modules/utils.py`	CLI argument parser and the Gaussian profile helper.
`example.py`	Hard-coded Ne VIII 770 Å time-series driver.
`example_CIII.py`	Same driver, but ion parameters come from the command line.
`scripts/example.sh`	PBS batch script for the NCAR/GLADE HPC environment.
`examineSynthesis.ipynb`	Notebook for loading and inspecting saved output.
`docs/introduction.md`	Extended prose walkthrough of the internals.

The clean separation of concerns is the code's main structural virtue: `muram.py` knows *file formats*, `dataLoader.py` knows *which variables synthesis needs*, and `synthesizeLines.py` knows *line physics*. The `example*.py` scripts are thin orchestrators.

3 Data Flow

Read	→	Configure	→	Synthesize
MURaM binaries `eosT`, `eosne`, `result_prim_*`		`MURaMData` → `Temp`, `ne`, `vz`		`spectrum` + CHIANTI $G(T, n_e)$ spline, λ grid

$\Longrightarrow$ per-column integration $\Longrightarrow$ `.npz` output (`I2D`, `wvlGrid`)

The outer time loop is parallelised over snapshots with `joblib.Parallel`; each worker loads one snapshot and returns one (n_x, n_y, n_λ) cube.

4 How Each Layer Works

Reading MURaM (`muram.py`).

`MuramCube` subclasses `numpy.memmap` so that a full 3D cube is never loaded into RAM unless sliced. A `Header.<iter>` file supplies the grid shape, cell sizes, domain size and simulation time. Because the binary storage order is (z, y, x), the reader opens the memmap with the reversed shape and transposes it back to a Python-facing (x, y, z) ordering (with x vertical). A `_varmap` dictionary maps friendly names (`rho`, `Temp`, `ne`, `Bx`, heating and current terms, ...) to file prefixes. `MuramSnap` simply attempts every known variable and records which files were `available`. Parallel `MuramSub*` classes handle subdomain output, and `MuramSlice`/`MuramColumn`/`MuramIntensity` handle 2D products.

Adapting the data (`dataLoader.py`).

`MURaMData.loadModel()` opens a snapshot and exposes exactly the three cubes the synthesis needs, each transposed: `Temp = muramModel.Temp.T`, `ne = muramModel.ne.T`, `vz = muramModel.vx.T`. The last axis after transposition is MURaM's vertical direction, i.e. the line-of-sight integration axis.

The contribution function (`spectralLine`).

For a chosen ion, `computeGtables()` builds a table of the line emissivity $G(T, n_e)$ over a log-spaced grid ($\log_{10} T \in [4, 7.5]$, $\log_{10} n_e \in [5, 12]$ by default) by calling ChiantiPy per density sample, then wraps it in a `RectBivariateSpline` for fast interpolation during synthesis. Table generation is itself parallelised over density samples.

Line synthesis (`spectrum`).

For a single column, `compute_spectrum1D` walks up in height and, at each cell, adds a normalised Gaussian weighted by $G(T, n_e)$:

$$\Delta\lambda_{\mathrm{Doppler}} = \frac{v_z}{c}\,\lambda_0, \qquad \mathrm{FWHM}_{\mathrm{thermal}} = \lambda_0\sqrt{\frac{8\ln 2\,k_B T}{m_{\mathrm{ion}}\,c^2}}, \quad (1)$$

$$I(\lambda) = \sum_z G(T_z, n_{e,z})\;\mathcal{G}(\lambda;\;\lambda_0 + \Delta\lambda_{\mathrm{Doppler}},\;\sigma_z)\,, \qquad \sigma_z = \frac{\mathrm{FWHM}_{\mathrm{thermal}}}{2.355}. \quad (2)$$

`compute_2Dspectrum` applies this to every (x, y) column. It first clips away the deep, hot convection-zone layers by finding, per column, the height nearest `TempClip` $= 2\times10^4$ K (searching only above a fixed `indexOffset = 150`), and integrates from there upward.

5 Running It

```
python example_CIII.py --ion c --ion_level 3 \
    --lambda0 977 --numWaves 200 --dLambda 0.015
```

Runtime configuration (`model_dir`, `workDir`, `tStart`/`tEnd`/`dTimeStep`, `downsample`, `n_jobs`) is edited directly in the driver script. Output is written by `numpy.savez` as an `.npz` file containing `I2D` (the intensity cube) and `wvlGrid` (the wavelength axis). An integrated line image for one snapshot is `I2D[t].sum(axis=2)`.

6 Code Review

The code is scientifically well-structured and readable. The review below is intended to harden it; findings are ordered by severity. Where a numeric factor is quoted, it was confirmed by a standalone calculation for the flagship Ne VIII 770 Å case.

Sev.	Location	Finding
High	`synthesizeLines.py` `compute_spectrum1D`	Thermal-width unit mismatch. c is CGS (`3e10` cm/s, consistent with the Doppler term where v_z is in cm/s), but k_B is SI (`1.380649e-23` J/K) and `ionMass` is in kg. Mixing SI k_B/mass with a cm/s c makes every thermal line width $\sim$**100$\times$ too narrow**. Fix: use $c = 3 \times 10^8$ m/s inside the thermal term (or switch k_B and mass to CGS).
High	`synthesizeLines.py` `spectrum.__init__`, `mass_dict`	The atomic-mass table mixes atomic *mass* and atomic *number*: `c`=12 and `mg`=24 are masses (correct), but `ne`=10, `o`=8, `n`=7, `fe`=26, `s`=16 are atomic *numbers*. The flagship Ne VIII case uses 10 instead of $\approx$20, widening its thermal profile by $\sqrt{2}$. (Net effect for Ne VIII, combined with the row above: $\sim$70$\times$ too narrow.) Also `"s":16` appears twice.
High	`synthesizeLines.py` `def_GTnfn` call	The saved-table *loading* path is unreachable. `spectrum.__init__` calls `def_GTnfn(self.loadGTables, self.GTablesFile)` *positionally*, but the signature is `def_GTnfn(TRange, neRange, loadGtable=False, gSaveFile=None)`. So `loadGtable` keeps its default `False` and the code always recomputes tables via CHIANTI, ignoring `GTablesFile`. Compounded by `self.temp, self.ne, self.GTne_table = np.load(gSaveFile)`, which cannot unpack an `NpzFile`; use `np.load(...)` keys (or `allow_pickle`) explicitly.
Medium	`dataLoader.py:19`	`loadModel(verbose=True)` prints `run1.available`; `run1` is undefined $\Rightarrow$ `NameError`. Should reference `self.muramModel.available`.
Medium	`dataLoader.py:23`	`self.vz = self.muramModel.vx.T`. Physically defensible (x is MURaM's vertical axis, and the integration is along the vertical line of sight), but the name `vz` sourced from `vx` is a trap. Add a comment stating the line-of-sight convention explicitly.
Medium	`example.py:92-115`	The trailing plotting loop is broken/leftover: `tStep` is undefined at module scope ($\Rightarrow$ `NameError`); indices like `I2D[el,100,100,el]` and `I2D[el,:,:,50]` conflate the timestep and wavelength axes; and `I2D` is pre-allocated as $(n_x, n_y, n_\lambda, n_t)$ but then overwritten by the `Parallel` result ordered $(n_t, n_x, n_y, n_\lambda)$. Delete or repair.
Low	`example*.py`	`nx = ny = ...shape[0]` assumes a square horizontal domain; use the true `shape[0]` and `shape[1]`.
Low	`synthesizeLines.py` `computeGtables`	Grid uses `np.arange(num)/num` rather than `/(num-1)`, so the endpoint (`Tmax`, `neMax`) is never actually reached (mild off-by-one in span).
Low	`synthesizeLines.py` `compute_spectrum1D`	`Gtne(T,ne)` is evaluated twice per cell (once in the `if`, once in the body). Cache it in a local to halve interpolation calls in the hot loop.
Low	Whole repo	Research-script rough edges: hard-coded HPC paths and `n_jobs`; per-column inner loop is serial (the `joblib` path is commented out); no `requirements.txt`/`environment.yml`; `modules/` is not an importable package (relies on `sys.path.append`); no tests.

Suggested next steps

1. Fix the thermal-width units and the atomic-mass table — these change the science output directly.
2. Repair (and unit-test) the $G(T, n_e)$ load path so precomputed tables can actually be reused.
3. Lift paths, time range, `downsample` and `n_jobs` into CLI arguments; add a dependency file

and package `modules/`.

4. Add a small synthetic-atmosphere regression test for `compute_spectrum1D`/`compute_2Dspectrum`.

Generated from a review of the `sinSPICE` source tree. Line references reflect the state of the working tree at review time.